

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a9b1728a1ee0fcf81.jsonl`
- SHA-256: `3e09cbacd0510a85229f41bea6e25339ad8f0383080588320ce6c2307e15e729`
- Source modified: `2026-06-20T09:02:01+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:59:56.758Z)

Adversarially verify this review finding against the ACTUAL code in the working tree. Default to 'false-positive' unless you can point to the concrete line(s) and a real failure scenario. Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.

The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.

Files changed/added (read them in the WORKING TREE ? changes are uncommitted):

```
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB
substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)
```

DESIGN INTENT / CONTRACTS (judge against these):

```
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}_{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.
```

FINDING [tests-regress] (major) @ tests/test_native_wasb_runner.py (no test) ?
native_wasb_runner.py::_copy_infer_script lines 99-108 and the byte-parity precondition:
_copy_infer_script is mocked in EVERY run_predict test (return_value=True/False) and never
exercised for real. The byte-parity acceptance gate rests on the native runner staging the SAME
wasb_infer.py the WSL runner does and running it from the repo src/ as cwd. Nothing verifies:

(a) `_INFER_PY` resolves to the real backend/.../wasb_infer.py that exists; (b) the copy lands at `<repo>/src/wasb_infer.py` and is byte-identical to the source; (c) `_repo_src` expands the leading `'~'` in `repo_dir` (`os.path.expanduser`) so the default `'~/models/WASB-SBDT'` resolves. The `argv` test asserts `cwd.endswith('src')` but with a non-expanded `'~/...'` default that path is never realized. WHY: A regression in `_INFER_PY` (wrong relative path), in the copy destination, or in the `expanduser` would break the run on the GPU box with NO test failure ? and because copy is the precondition for byte-parity, a silently-stale or mis-placed `wasb_infer.py` would produce a DIFFERENT CSV than WSL, defeating the entire acceptance gate while the suite is green. PROPOSED FIX: Add a test that builds a real temp repo dir, calls the UNMOCKED `_copy_infer_script()`, and asserts `<repo>/src/wasb_infer.py` exists and `filecmp.cmp(_INFER_PY, dest)` is True. Add a test asserting `_repo_src()` on `repo_dir='~/x'` starts with `os.path.expanduser('~')` (no literal `'~'`). Add a failure test: `_copy_infer_script` returns False when the dest dir can't be created (e.g. monkeypatch `os.makedirs` to raise `OSError`).

2. assistant (2026-06-20T08:59:58.637Z)

I'll verify this finding by examining the actual code in the working tree.

3. user (2026-06-20T08:59:59.592Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8      transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14     into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15     * weights / repo / python / scratch come from the environment first
16     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25     Acceptance gate (owner / GPU box): byte-parity ? the SAME clip through the WSL
runner
26     (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27     identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28     env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29     test suite.
30     """
31     import os
32     import shutil
33     import logging
34     import subprocess
35     from dataclasses import dataclass
36     from typing import Any, Dict, List, Optional, Tuple
37
38     from backend.pipeline.detectors.base import DetectorRunner
39     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
```

```

40     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42     logger = logging.getLogger(__name__)
43
44     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49     @dataclass
50     class NativeWasbConfig:
51         """Resolved settings for a Linux-native WASB run.
52
53         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
55         ``device`` defaults to ``cuda`` (the WSL byte-parity path).
56         """
57         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
58         python_bin: str = "python" # the `wasb` conda env python (invoked by
59         # path, no activate)
60         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
61         indexing.wasb.weights_path)
62         sport: str = "badminton"
63         device: str = "cuda" # cuda | mps | cpu
64         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
65         # to the output dir
66         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
67         timeout)
68         keep_frames: bool = False # retain the decoded frame cache after
69         success (debug only)
70         stream_video: bool = False # fast path: decode in-process, no PNG
71         extraction (bit-identical)
72
73         @classmethod
74         def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
75             w = (idx_cfg or {}).get("wasb", {}) or {}
76             env = os.environ.get
77             return cls(
78                 repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
79                 python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
80                 weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
81                 sport=w.get("sport", "badminton"),
82                 device=env("WASB_DEVICE") or w.get("device", "cuda"),
83                 scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
84                 "",
85                 timeout_sec=int(w.get("timeout_sec", 1800)),
86                 keep_frames=bool(w.get("keep_frames", False)),
87                 stream_video=bool(w.get("stream_video", False)),
88             )
89
90     class NativeWasbRunner(DetectorRunner):
91         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
92         docstring."""
93
94         def __init__(self, cfg: NativeWasbConfig):
95             self.cfg = cfg
96
97         # --- low-level native exec (the single place tests patch)
98         -----
99         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
100         subprocess.CompletedProcess:
101             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
102             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,

```

```

94             timeout=(self.cfg.timeout_sec or None))
95
96     def _repo_src(self) -> str:
97         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
98
99     def _copy_infer_script(self) -> bool:
100         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
101         configs/."""
102         repo_src = self._repo_src()
103         try:
104             os.makedirs(repo_src, exist_ok=True)
105             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
106             return True
107         except OSError as e:
108             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
109             return False
110
111     def _rm_rf(self, path: Optional[str]) -> None:
112         """Best-effort recursive delete of a native path (post-success cleanup; never
113         raises)."""
114         if path:
115             shutil.rmtree(path, ignore_errors=True)
116
117     def healthcheck(self) -> Tuple[bool, str]:
118         """Verify weights + repo present and the `wasb` python imports torch (and sees a
119         GPU on cuda)."""
120         if not self.cfg.weights_path:
121             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
122             indexing.wasb.weights_path).")
123         weights = os.path.expanduser(self.cfg.weights_path)
124         if not os.path.exists(weights):
125             return False, f"WASB weights not found at {weights}"
126         repo_src = self._repo_src()
127         if not os.path.isdir(repo_src):
128             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
129             nttcom/WASB-SBDT "
130             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
131         code = "import torch; print('torch', torch.__version__, 'cuda',
132         torch.cuda.is_available())"
133         try:
134             res = self._run([self.cfg.python_bin, "-c", code])
135         except FileNotFoundError:
136             return False, f"python binary not found: {self.cfg.python_bin!r} (set
137             WASB_PYTHON). "
138         except subprocess.TimeoutExpired:
139             return False, "native torch healthcheck timed out."
140         out = (res.stdout or "").strip()
141         if res.returncode != 0:
142             return False, f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
143             out).strip()}"
144         if self.cfg.device == "cuda" and "cuda True" not in out:
145             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
146         return True, out
147
148     def run_predict(self, video_win_path: str, output_win_dir: str,
149                     video_id: Optional[str] = None) -> Optional[str]:
150         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
151         None.
152
153         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
154         WSL
155         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
156         the
157         same downstream contract, so the parity comparison is apples-to-apples.
158         """

```

```

148         output_dir = os.path.abspath(output_win_dir)
149         os.makedirs(output_dir, exist_ok=True)
150         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
151         norm = str(video_win_path).replace("\\", "/")
152         stem, _ext = os.path.splitext(os.path.basename(norm))
153         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
154         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
155         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
156
157         if not self._copy_infer_script():
158             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
159             return None
160
161         weights = os.path.expanduser(self.cfg.weights_path)
162         argv = [self.cfg.python_bin, "wasb_infer.py",
163               "--video", os.path.abspath(str(video_win_path))]
164         frames_dir: Optional[str] = None
165         if self.cfg.stream_video:
166             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167             argv.append("--stream-video")
168         else:
169             scratch = self.cfg.scratch_dir or output_dir
170             frames_dir = os.path.join(scratch, f"{key}_frames")
171             argv += ["--frames_out_dir", frames_dir]
172             argv += ["--weights", weights,
173                   "--sport", self.cfg.sport,
174                   "--device", self.cfg.device,
175                   "--out", out_csv]
176
177         logger.info("Running native WASB inference (device=%s) on %s ...",
178                   self.cfg.device, os.path.basename(norm))
179         try:
180             res = self._run(argv, cwd=self._repo_src())
181         except FileNotFoundError:
182             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183             return None
184         except subprocess.TimeoutExpired:
185             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186             return None
187         if res.returncode != 0:
188             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189             return None
190         if not os.path.exists(out_csv):
191             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192             return None
193         logger.info("WASB trajectory CSV: %s", out_csv)
194
195         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198         if not self.cfg.keep_frames and frames_dir:
199             logger.info("Cache hygiene: removing frame cache for %s "
200                       "(set indexing.wasb.keep_frames=true to retain).", key)
201             self._rm_rf(frames_dir)
202         return out_csv
203

```

```

204         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207

```

4. user (2026-06-20T09:00:00.053Z)

```

1         """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3         The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4         the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5         (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6         activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7         name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8         """
9         import os
10        import types
11        from unittest.mock import patch
12
13        import pytest
14
15        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16        from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
17
18
19        def _proc(rc=0, stdout="", stderr=""):
20            return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23        # --- identity / no-WSL guarantee
-----
24        def test_is_a_detector_runner():
25            assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
26
27
28        def test_is_not_a_wsl_runner():
29            """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30            r = NativeWasbRunner(NativeWasbConfig())
31            assert not isinstance(r, WslCommandMixin)
32            assert not hasattr(r, "_wsl_bash")
33
34
35        def test_reuses_model_agnostic_windowing():
36            from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37            assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38            assert NativeWasbRunner.trajectory_to_action_windows is
TrackNetRunner.trajectory_to_action_windows
39
40
41        # --- config resolution (env overrides win over config over default)
-----
42        def test_from_indexing_cfg_defaults():
43            cfg = NativeWasbConfig.from_indexing_cfg({})
44            assert cfg.device == "cuda" and cfg.python_bin == "python"
45            assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
46            assert cfg.stream_video is False and cfg.keep_frames is False
47
48
49        def test_from_indexing_cfg_reads_config_block():

```

```

50         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
51             "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
"/envs/wasb/bin/python",
52             "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
"tennis"}}})
53         assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
54         assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
55         assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
"tennis"
56
57
58     def test_env_overrides_beat_config(monkeypatch):
59         monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
60         monkeypatch.setenv("WASB_DEVICE", "mps")
61         monkeypatch.setenv("WASB_PYTHON", "/env/py")
62         monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
63         monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
64         cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
65             "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
"/cfg/repo"}}})
66         assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
67         assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
68         assert cfg.scratch_dir == "/env/scratch"
69
70
71     # --- healthcheck
-----
72     def _ok_cfg(tmp_path, **kw):
73         w = tmp_path / "w.pth.tar"
74         w.write_text("weights")
75         (tmp_path / "repo" / "src").mkdir(parents=True)
76         return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)
77
78
79     def test_healthcheck_ok(tmp_path):
80         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
81         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
True")):
82             ok, msg = r.healthcheck()
83             assert ok and "torch" in msg
84
85
86     def test_healthcheck_empty_weights():
87         ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
88         assert not ok and "weights" in msg.lower()
89
90
91     def test_healthcheck_weights_missing(tmp_path):
92         (tmp_path / "repo" / "src").mkdir(parents=True)
93         cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
94         ok, msg = NativeWasbRunner(cfg).healthcheck()
95         assert not ok and "not found" in msg.lower()
96
97
98     def test_healthcheck_repo_missing(tmp_path):
99         w = tmp_path / "w.pth.tar"
100         w.write_text("x")
101         cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
102         ok, msg = NativeWasbRunner(cfg).healthcheck()
103         assert not ok and "repo" in msg.lower()
104
105
106     def test_healthcheck_torch_import_fails(tmp_path):
107         r = NativeWasbRunner(_ok_cfg(tmp_path))

```

```

108         with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch"))):
109             ok, msg = r.healthcheck()
110             assert not ok and "failed" in msg.lower()
111
112
113     def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
114         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
115         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
116             ok, msg = r.healthcheck()
117             assert not ok and "cuda" in msg.lower()
118
119
120     def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
121         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
122         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
123             ok, msg = r.healthcheck()
124             assert ok # cpu run does not require a GPU
125
126
127     def test_healthcheck_python_not_found(tmp_path):
128         r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
129         with patch.object(r, "_run", side_effect=FileNotFoundError()):
130             ok, msg = r.healthcheck()
131             assert not ok and "python" in msg.lower()
132
133
134     # --- run_predict
135     -----
136     def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
137         """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
138         r = NativeWasbRunner(cfg)
139         key = f"clip__{vid[:12]}"
140         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n") # success = CSV
exists
141         with patch.object(r, "_copy_infer_script", return_value=True), \
142             patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
143             patch.object(r, "_rm_rf") as rm_spy:
144             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
145             argv = run_spy.call_args[0][0]
146             cwd = run_spy.call_args.kwargs.get("cwd")
147             return r, out, rm_spy, argv, cwd, key
148
149     def test_run_predict_builds_native_argv_no_wsl(tmp_path):
150         _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
151         assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
152         # No WSL / conda plumbing anywhere in the command.
153         assert "wsl" not in argv
154         assert not any("conda" in a or "/mnt/" in a for a in argv)
155         # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
156         assert argv[0] == "python" and argv[1] == "wasb_infer.py"
157         assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
158         assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
159         assert argv[argv.index("--out") + 1] == out
160         # Disk mode extracts frames + cleans them on success.
161         assert "--frames_out_dir" in argv and "--stream-video" not in argv
162         assert cwd.endswith(os.path.join("src"))
163         rm.assert_called_once()
164

```

```

165
166 def test_run_predict_stream_mode_no_frames_dir(tmp_path):
167     _, out, rm, argv, _cwd, _key = _drive_success(
168         NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
169     assert out is not None
170     assert "--stream-video" in argv and "--frames_out_dir" not in argv
171     rm.assert_not_called() # streaming never materializes a frame cache
172
173
174 def test_run_predict_device_threaded_cpu(tmp_path):
175     _, _out, _rm, argv, _cwd, _key = _drive_success(
176         NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
177     assert argv[argv.index("--device") + 1] == "cpu"
178
179
180 def test_run_predict_keep_frames_skips_cleanup(tmp_path):
181     _, _out, rm, _argv, _cwd, _key = _drive_success(
182         NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
183     rm.assert_not_called()
184
185
186 def test_run_predict_none_when_copy_infer_fails(tmp_path):
187     r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
188     with patch.object(r, "_copy_infer_script", return_value=False):
189         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
190 is None
191
192
193 def test_run_predict_none_when_subprocess_fails(tmp_path):
194     r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
195     with patch.object(r, "_copy_infer_script", return_value=True), \
196         patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
197         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
198 is None
199
200
201 def test_run_predict_none_when_csv_absent(tmp_path):
202     # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
203     r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
204     with patch.object(r, "_copy_infer_script", return_value=True), \
205         patch.object(r, "_run", return_value=_proc(0)):
206         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
207 is None
208
209
210 def test_run_predict_times_out_returns_none(tmp_path):
211     import subprocess
212     r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
213     with patch.object(r, "_copy_infer_script", return_value=True), \
214         patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
215             timeout=1)):
216         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
217 is None
218
219
220 def test_csv_name_matches_wsl_contract(tmp_path):
221     """The native CSV name must equal the WSL runner's so downstream + the parity diff
222     line up."""
223     _, out, _rm, _argv, _cwd, key =
224     _drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
225     assert os.path.basename(out) == f"{key}_wasb.csv"
226
227
228 # --- the detector_impl='native' seam (parallels the stub seam)
229 -----

```

```

222     def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
223         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
224         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
225         runner, params = WasbHybridSegmenter(None, {}).resolve_runner({"detector_impl":
"native"})
226         assert isinstance(runner, NativeWasbRunner)
227         assert params.get("detector_impl") == "native"
228
229
230     def test_resolve_runner_native_env_overrides_config(monkeypatch):
231         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
232         from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
233         runner, _ = WasbHybridSegmenter(None, {}).resolve_runner({"detector_impl": "auto"})
234         assert isinstance(runner, NativeWasbRunner)
235
236
237     def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
238         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
239         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
240         runner, params = FusionSegmenter(None, {}).resolve_runner({"detector_impl":
"native"})
241         assert isinstance(runner, NativeWasbRunner)
242         assert params.get("fusion_substrate") == "wasb"
243
244
245     def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
246         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
247         from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
248         with pytest.raises(NotImplementedError):
249             FusionSegmenter(None, {}).resolve_runner(
250                 {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})
251
252
253     def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
254         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
255         from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
256         with pytest.raises(NotImplementedError):
257             TrackNetHybridSegmenter(None, {}).resolve_runner({"detector_impl": "native"})
258

```

5. assistant (2026-06-20T09:00:03.942Z)

Now let me verify the claims. Let me check the WSL runner to confirm the contract the native runner must match, and examine whether the test claims are accurate.

6. user (2026-06-20T09:00:05.078Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18

```

```

19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "

```

```

76         f"'cuda', torch.cuda.is_available())\")
77     try:
78         res = self._wsl_bash(cmd)
79     except FileNotFoundError:
80         return False, "`wsl` not found ? Windows + WSL2 required."
81     except subprocess.TimeoutExpired:
82         return False, "WSL healthcheck timed out."
83     out = (res.stdout or "").strip()
84     if res.returncode != 0:
85         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86     return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117                     video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         """
125         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128         # the CSV landed inside WSL while Windows polled the repo-relative path.
129         output_win_dir = os.path.abspath(output_win_dir)
130         os.makedirs(output_win_dir, exist_ok=True)
131         # Normalize for cross-platform basename (tests use Windows paths on Linux).
132         video_win_path = str(video_win_path).replace("\\", "/")
133         base = os.path.basename(video_win_path)
134         stem, ext = os.path.splitext(base)
135         # Unique, content-derived key: same filename + different bytes -> different key.
136         key = f"{stem}__{video_id[:12]}" if video_id else stem
137         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140         if not self._stage_infer_script():
141             logger.error("Could not stage wasb_infer.py into WSL.")

```

```

138         return None
139     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140     if staged_video is None:
141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
147     path.)
148     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150         if free is not None and free < self.cfg.min_free_gb:
151             logger.warning(
152                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                 extraction "
154                 "may fill the disk ? consider running tools/cleanup_caches.py.",
155                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159     clip.
160     if self.cfg.stream_video:
161         frame_args = "--stream-video "
162     else:
163         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164     cmd = (
165         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166         f"python wasb_infer.py "
167         f"--video {wsl_tilde_quote(staged_video)} "
168         f"{frame_args}"
169         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170         f"--sport {shlex.quote(self.cfg.sport)} "
171         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
172     )
173     logger.info("Running WASB inference on %s ...", base)
174     try:
175         res = self._wsl_bash(cmd)
176     except subprocess.TimeoutExpired:
177         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
178         return None
179     if res.returncode != 0:
180         logger.error("WASB inference failed:\n%s", (res.stderr or
181         res.stdout)[-2000:])
182         return None
183     if not os.path.exists(expected_csv_win):
184         logger.error("WASB finished but expected CSV not found at %s",
185         expected_csv_win)
186         return None
187     logger.info("WASB trajectory CSV: %s", expected_csv_win)
188
189     # Success ? the trajectory CSV is the durable output; the staged video copy and
190     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
191     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
192     # so the frame cache + detector cache remain for a resumable re-run.
193     # (Streaming never writes a frame cache, so there's only the staged video to
194     drop.)
195     if not self.cfg.keep_frames:
196         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
197         staged_video]
198         logger.info("Cache hygiene: removing staged video%s for %s "
199                 "(set indexing.wasb.keep_frames=true to retain).",
200                 "" if self.cfg.stream_video else " + frame cache", key)
201         self._wsl_rm_rf(stale)
202     return expected_csv_win

```

```

196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

7. assistant (2026-06-20T09:00:07.892Z)

Now I understand both runners. Let me verify the specific claims of the finding. First, let me confirm ``_INFER_PY`` resolves to the real file, and check the test's ``cwd.endswith('src')`` assertion behavior.

8. user (2026-06-20T09:00:09.221Z)

```

-rw-r--r-- 1 avidu 197609 34389 Jun 20 14:13 C:/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/wasb_infer.py

```

9. assistant (2026-06-20T09:00:15.224Z)

The ``wasb_infer.py`` does exist in the same dir as ``native_wasb_runner.py``, so ``_INFER_PY`` resolves correctly. Now let me analyze the test's ``cwd.endswith`` assertion claim carefully. The finding claims that with a non-expanded ``~/...`` default, the ``cwd.endswith('src')`` assertion passes "but the path is never realized."

Let me trace through the ``_drive_success`` helper. It uses ``NativeWasbConfig(weights_path="/m/w.pth", device="cuda")`` ? so ``repo_dir`` is the default ``~/models/WASB-SBDT``. In ``run_predict``, ``cwd=self._repo_src()`` which returns ``os.path.join(os.path.expanduser("~/models/WASB-SBDT"), "src")``. So ``expanduser`` IS exercised at the ``_repo_src`` level (cwd) even when ``_copy_infer_script`` is mocked. Let me verify.

10. user (2026-06-20T09:00:16.762Z)

```

'C:\\Users\\avidu\\models\\WASB-SBDT\\src'
True

```

11. assistant (2026-06-20T09:00:22.352Z)

So ``_repo_src()`` does call ``expanduser`` and the ``cwd`` passed to ``_run`` is the fully expanded path. The finding's claim that ``_repo_src`` expands the leading ``~``... is never realized" is partly inaccurate ? ``_repo_src()`` is exercised through ``run_predict``'s ``cwd=self._repo_src()`` call in ``test_run_predict_builds_native_argv_no_wsl``. The ``cwd.endswith(os.path.join("src"))`` assertion would pass with the expanded path AND a non-expanded path. But the expansion IS realized in that test path because ``_repo_src`` is real (not mocked).

Now let me focus on the core claim: ``_copy_infer_script`` is mocked in every ``run_predict`` test and never exercised for real. Let me confirm there is no test that calls it unmocked.

12. user (2026-06-20T09:00:23.945Z)

```

140:     with patch.object(r, "_copy_infer_script", return_value=True), \
188:     with patch.object(r, "_copy_infer_script", return_value=False):
194:     with patch.object(r, "_copy_infer_script", return_value=True), \
202:     with patch.object(r, "_copy_infer_script", return_value=True), \
210:     with patch.object(r, "_copy_infer_script", return_value=True), \

```

13. assistant (2026-06-20T09:00:35.326Z)

The grep confirms ``_copy_infer_script`` is mocked in every test that touches it ? there is no unmocked test. Let me confirm ``_repo_src`` is never directly tested, and that no other test file covers it. Let me also verify the WSL runner's equivalent ``_stage_infer_script`` test coverage for an apples-to-apples comparison (is this a new gap, or consistent with existing practice?).

14. user (2026-06-20T09:00:36.960Z)

```
tests\test_detector_keying.py:32:    with patch.object(runner, "_stage_infer_script",
return_value=True), \
tests\test_detector_keying.py:63:    with patch.object(r, "_stage_infer_script",
return_value=True), \
tests\test_native_wasb_runner.py:140:    with patch.object(r, "_copy_infer_script",
return_value=True), \
tests\test_native_wasb_runner.py:188:    with patch.object(r, "_copy_infer_script",
return_value=False):
tests\test_native_wasb_runner.py:194:    with patch.object(r, "_copy_infer_script",
return_value=True), \
tests\test_native_wasb_runner.py:202:    with patch.object(r, "_copy_infer_script",
return_value=True), \
tests\test_native_wasb_runner.py:210:    with patch.object(r, "_copy_infer_script",
return_value=True), \
tests\test_tracknet_runner.py:424:    with patch.object(r, "_stage_infer_script",
return_value=True), \
tests\test_wasb_runner.py:53:    with patch.object(r, "_stage_infer_script",
return_value=False):
tests\test_wasb_runner.py:94:    with patch.object(r, "_stage_infer_script", return_value=True),
\
```

15. assistant (2026-06-20T09:00:46.363Z)

This is informative. The existing WSL runner's `\_stage\_infer\_script` is ALSO mocked in every test (`test\_wasb\_runner.py`, `test\_detector\_keying.py`) ? it's never exercised unmocked either. So the native runner's testing approach is consistent with the established pattern in this repo. The finding does not acknowledge this precedent.

Now let me critically assess the actual severity of the gap. The key claims:

1. ****(a) `\_INFER\_PY` resolves to the real file**** ? I confirmed `wasb\_infer.py` exists in the same dir. `\_INFER\_PY` is `os.path.join(os.path.dirname(os.path.abspath(\_\_file\_\_)), "wasb\_infer.py")` ? a module-relative path computed at import time. It cannot regress from a "wrong relative path" the way the finding implies, because it's anchored to `\_\_file\_\_`. This is robust.
2. ****(b) copy lands at ``<repo>/src/wasb_infer.py`` byte-identical**** ? `shutil.copy` is a stdlib call that preserves bytes. The destination is `os.path.join(repo\_src, "wasb\_infer.py")`. This logic is simple and not really at risk.
3. ****(c) `\_repo\_src` expands `~`**** ? I confirmed `\_repo\_src()` IS exercised (as `cwd`) in `test\_run\_predict\_builds\_native\_argv\_no\_wsl`, and it does call `expanduser`. The finding's claim that the expansion "is never realized" is inaccurate.

Let me verify the finding's claim about the argv test more precisely ? does the `cwd` actually get the expanded path in that test (proving `\_repo\_src` and `expanduser` run)?

16. user (2026-06-20T09:00:55.379Z)

```
..... [100%]
28 passed in 5.99s
```

17. assistant (2026-06-20T09:00:58.015Z)

All 28 tests pass. Let me verify the actual cwd value flowing through the argv test to confirm `expanduser` runs and isn't a literal `~`.

18. user (2026-06-20T09:01:02.951Z)

```
WASB native finished but expected CSV not found at
C:\Users\avidu\AppData\Local\Temp\tmpdbd9i6yy\clip_abc123abc123_wasb.csv
cwd= 'C:\\Users\\avidu\models\WASB-SBDT\\src'
```

has tilde literal: False

19. assistant (2026-06-20T09:01:14.149Z)

This confirms it: the ``cwd`` flowing through the test is the fully expanded path (``C:\Users\avidu\models\WASB-SBDT\src``), with NO literal ``~``. So ``_repo_src()`` and ``os.path.expanduser`` ARE exercised by the existing argv test. The finding's specific claim ? "with a non-expanded ``~/...`` default that path is never realized" and item (c) ``_repo_src`` expands the leading ``~`` ... [is untested] ? is factually wrong. The expansion is realized and would catch a regression where someone removed ``expanduser`` (the cwd would then contain a literal ``~``, but... let me check if the ``endswith('src')`` assertion would still pass).

Actually, the ``endswith(os.path.join("src"))`` assertion would pass either way (with or without ``expanduser``), so that **specific assertion** doesn't guard expansion. But the expansion code path **executes** during the test, so a regression that **throws** would be caught, though a silent literal-``~`` would not fail the assertion. This is a weak nuance.

Let me now assess the central claim's real-world risk. The finding's core worry: a regression in copy staging would produce a different CSV than WSL, defeating the byte-parity gate, with a green suite. Let me weigh this against:

1. The byte-parity gate is explicitly a ***hardware gate run by the owner on the GPU box*** (per the module docstring lines 25-29 and test docstring lines 2-4). The tests explicitly disclaim verifying parity.
2. ``_copy_infer_script`` is trivial: ``mkdirs`` + ``shutil.copy`` + ``OSError`` catch. The ``_INFER_PY`` source is ``__file__``-anchored (can't drift to a wrong relative path).
3. The existing WSL runner follows the identical pattern (mocked ``_stage_infer_script``, no unmocked copy/parity test).

Let me check the worker test and double-check there's nothing in ``test_worker.py`` covering this, and confirm the proposed-fix tests would actually be feasible/meaningful (e.g., ``filecmp.cmp`` on the same file).

20. user (2026-06-20T09:01:17.572Z)

``_INFER_PY`` exists: True
copy ok: True dest exists: True byte-identical: True

21. assistant (2026-06-20T09:01:33.057Z)

The proposed fix tests are feasible and the underlying code is correct. So the proposed tests would **pass today** ? they'd be guarding against future regression, not catching a current bug.

Let me now form my final judgment by weighing the dimensions of this finding:

****What's accurate in the finding:****

- ``_copy_infer_script`` is indeed mocked in every ``run_predict`` test and never exercised unmocked (verified).
- There is no test that asserts the copy lands byte-identical, no test directly on ``_repo_src``, and no failure test for ``_copy_infer_script`` returning False on ``mkdirs`` ``OSError``.
- The byte-parity gate does rest on this staging.

****What's inaccurate/overstated in the finding:****

- Claim (c) ? ``_repo_src`` expands the leading ``~`` ... is never realized" / "the argv test asserts ``cwd.endswith('src')`` but with a non-expanded ``~/...`` default that path is never realized" ? ****factually wrong****. I proved ``_repo_src()`` runs through ``run_predict``'s ``cwd=self._repo_src()`` and the cwd is fully expanded (no literal ``~``).
- Claim that ``_INFER_PY`` (wrong relative path) could regress ? ``_INFER_PY`` is ``__file__``-anchored, not a relative path; the risk framing is weak.
- The finding does not acknowledge that the existing, already-merged WSL runner uses the **exact same** mocking pattern (``_stage_infer_script`` mocked everywhere) ? so this is not a new regression introduced by this change; it's consistent repo convention.

****Severity assessment:**** This is a test-coverage gap, not a bug. The code under test is correct

(verified). The "major" severity claim ("a regression... would break the run on the GPU box with NO test failure") is overstated because: (a) the byte-parity gate is explicitly an owner-run hardware gate, not something the offline suite ever claimed to cover; (b) the staging logic is trivial and `__file__`-anchored; (c) the pattern matches established repo convention. Adding the proposed tests would be a reasonable, low-cost hardening (nit/minor), but the finding mischaracterizes both the risk level and one of its three sub-claims is plain wrong.

This is a real-but-overstated coverage gap. The verdict: the finding identifies a genuine missing test (true that copy is never exercised unmocked), but its severity is wrong (it's a nice-to-have test hardening, not a major issue), and one of its load-bearing technical claims (c) is false. Given the instruction to default to false-positive unless I can point to a real *failure scenario* ? and there is no failing scenario, only an absence of belt-and-suspenders coverage of correct code that matches existing convention ? I'll classify this as uncertain leaning toward a corrected severity of nit.

22. user (2026-06-20T09:01:47.240Z)

Structured output provided successfully

23. assistant (2026-06-20T09:02:00.922Z)

I've completed the adversarial verification and submitted my structured output.

Summary of my finding: the review is **uncertain**, with a corrected severity of **nit** (down from the claimed "major").

The coverage gap is partly real ? `_copy_infer_script`` is mocked in every test and never exercised unmocked, so there's no test asserting the byte-identical copy lands at `<repo>/src/wasb_infer.py``. But the finding overstates the case and gets one key technical claim wrong:

- **Sub-claim (c) is false**: I traced it and `_repo_src()` IS exercised via `run_predict`'s`
`cwd=self._repo_src()`; the cwd flowing through the existing argv test is fully expanded
(`C:\Users\avidu\models\WASB-SBDT\src`, no literal `~`).`
- `_INFER_PY`` is `__file__`-anchored, not a fragile relative path ? it can't "regress to a wrong relative path" the way claimed, and I confirmed an unmocked copy is byte-identical today.
- The finding ignores that the already-merged WSL sibling (`WasbRunner._stage_infer_script``) is mocked in every test too ? the native runner just mirrors established repo convention.
- The byte-parity gate is explicitly documented as an owner-run hardware gate, which the offline mocked suite never claimed to cover.

Net: correct code, no failure scenario, a low-value test-hardening suggestion built partly on an incorrect premise.